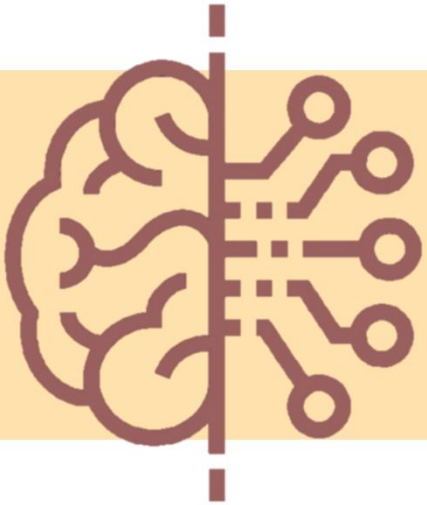
UTM
UNIVERSITI TEKNOLOGI MALAYSIA

www.utm.my

Heuristic Search Algorithm

(Informed Search)



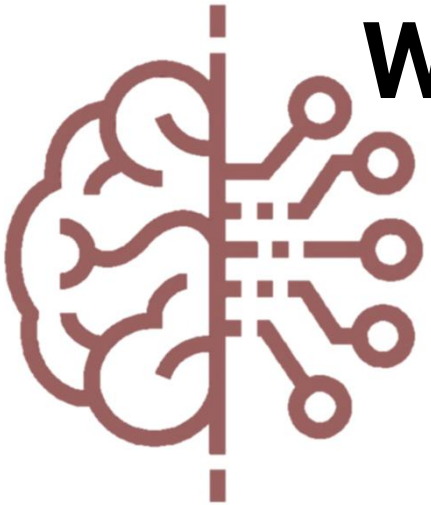
Artificial Intelligence

School of Computing
Universiti Teknologi Malaysia

Premier Digital Tech University ™

Outline

1. What is Heuristic Search?
2. Selection of a Search Strategy
   - Hill climbing Search
   - Best-first Search
   - Search with Heuristic
   - Comparison of all search strategy
3. Heuristic Evaluation Function f(n)
4. Exercise

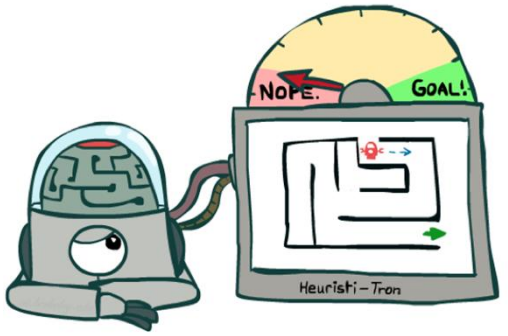


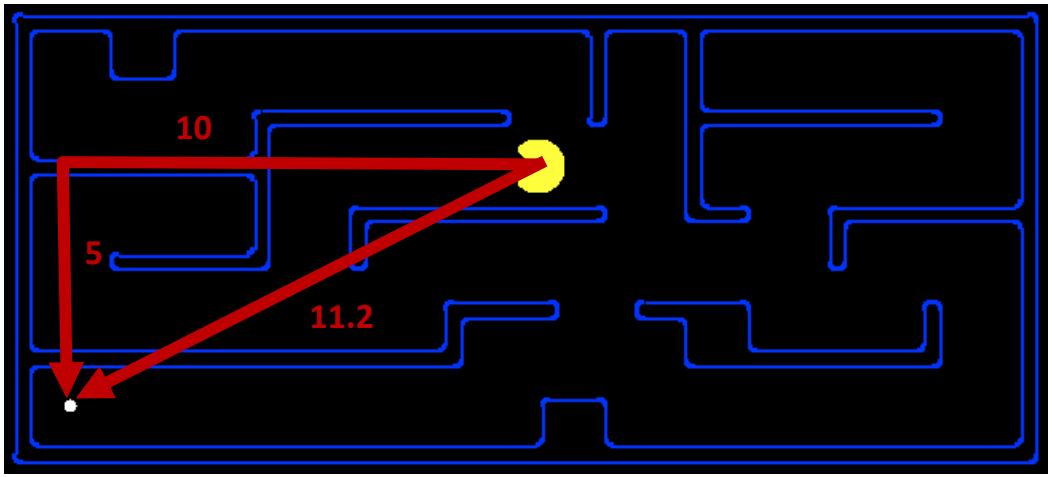
What is Heuristic Search?

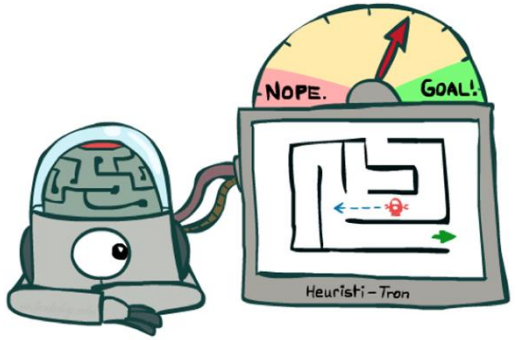
- Definition
- Taxonomy of Heuristic Search

Heuristic Search: Definition

- A heuristic is:
  - A function that *estimates* how close a state is to a goal
  - Designed for a particular search problem
  - Examples: Manhattan distance, Euclidean distance for path







Heuristic Search: Definition

- A technique to solve a problem faster than classic methods, or to find an approximate solution when classic methods cannot.
- Heuristic is a rule of thumb that probably leads to a solution. Heuristics play a major role in search strategies because of exponential nature of the most problems. Heuristics help to reduce the number of alternatives from an exponential number to a polynomial number
- This is a kind of a shortcut as we often trade one of optimality, completeness, accuracy, or precision for speed.
- A Heuristic (or a heuristic function) takes a look at search algorithms. At each branching step, it evaluates the available information and makes a decision on which branch to follow. It does so by ranking alternatives. The Heuristic is any device that is often effective but will not guarantee work in every case.

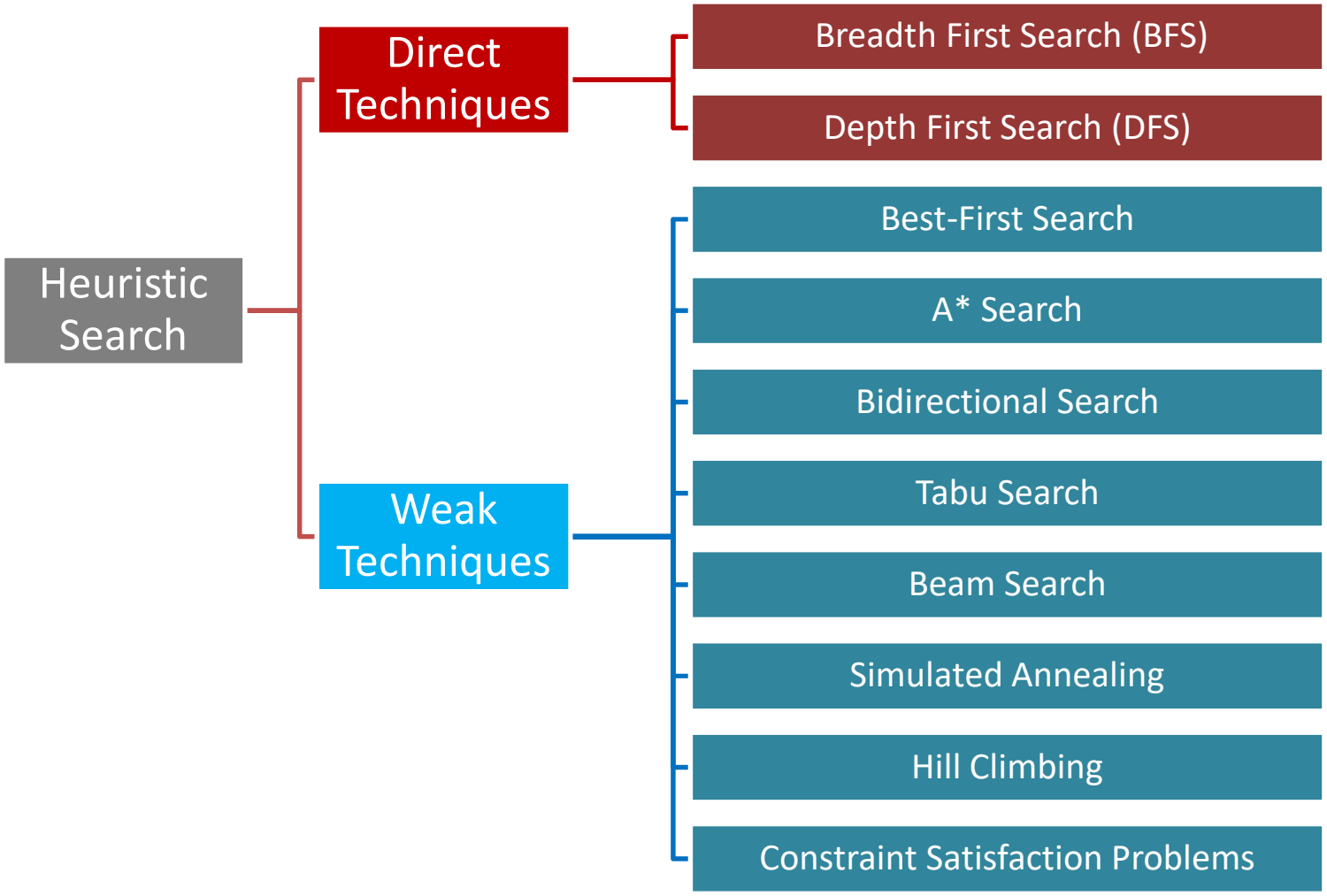
Heuristic Search: Definition

- (rules of thumb): Weak search method because it is based on experience or intuition.
- Have long been a core concern in AI research
- Used to prune spaces of possible solution
- When to employ Heuristic?
  1. A problem may not have an exact solution.

     e.g. medical diagnosis: doctors use heuristic
  2. A problem may have an exact solution, but the computational cost of finding it may be prohibitive.

     e.g in chess (exhaustive or brute force search)

Heuristic Search: Taxonomy



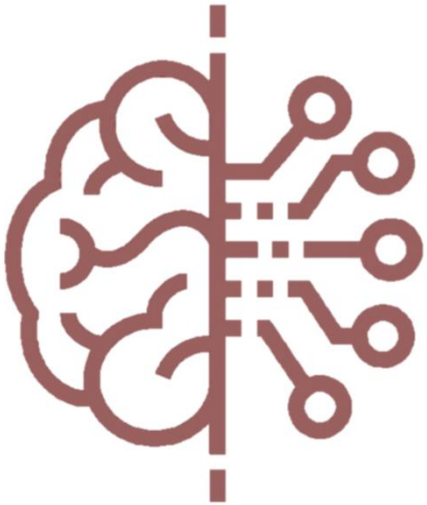
Heuristic Search: Taxonomy

Direct Heuristic Search Techniques

- Other names for these are Brute Force Search, Blind Search, Uninformed Search, and Blind Control Strategy.
- These aren't always possible since they demand much time or memory. They search the entire state space for a solution and use an arbitrary ordering of operations.
- Examples of these are Breadth First Search (BFS) and Depth First Search (DFS).

Weak Heuristic Search Techniques

- Other names for these are Informed Search, Heuristic Search, and Heuristic Control Strategy
- These are effective if applied correctly to the right types of tasks and usually demand domain-specific information.
- This extra information is needed to compute preference among child nodes to explore and expand. Each node has a heuristic function associated with it.
- Examples are Best First Search (BFS) and A*.



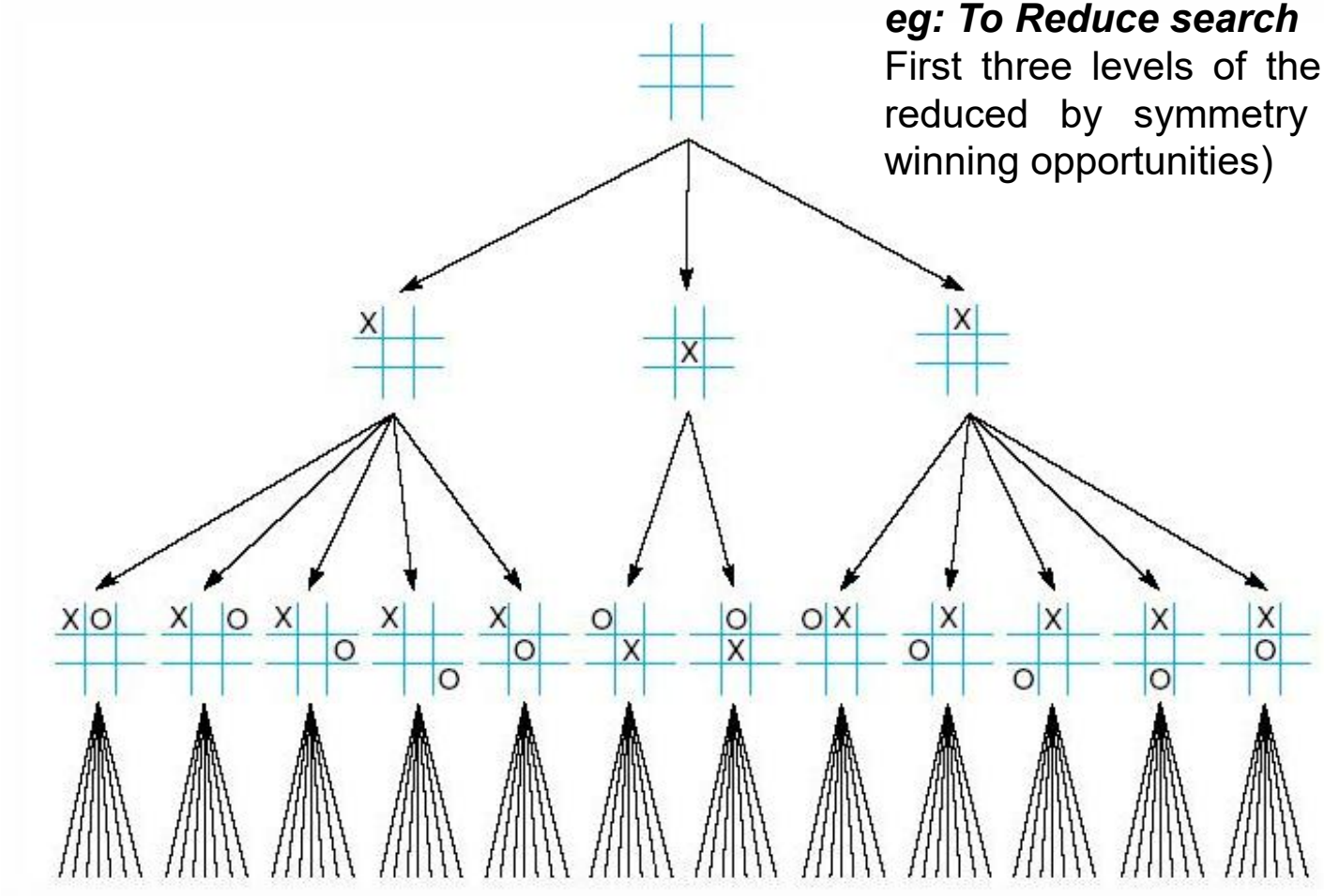
Direct Heuristic Search

- Introduction
- Anagram
- Breadth First Search (BFS)
- Depth First Search (DFS)
- Quiz

Direct Heuristic Search: Intro

- Other names for these are brute force search, blind search, uninformed search, and blind control strategy
- In computer science, a brute force search consists of systematically enumerating every possible solution of a problem until a solution is found, or all possible solutions have been exhausted.
- For example, an anagram problem can be solved by enumerating all possible combinations of words with the same number of letters as the desired phrase, and checking one by one whether the words make a valid anagram.

Direct Heuristic Search: Intro

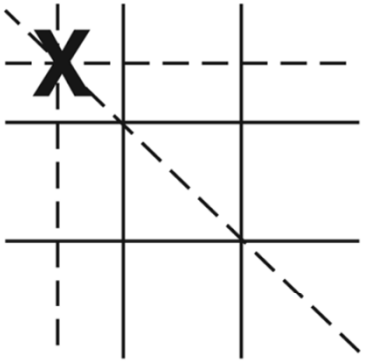


eg: To Reduce search

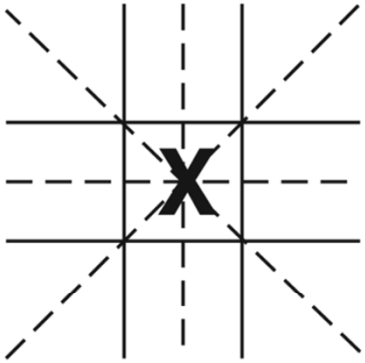
First three levels of the tic-tac-toe state space reduced by symmetry (simple heuristic-most winning opportunities)

Direct Heuristic Search: Intro

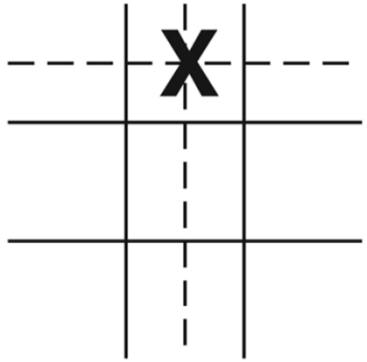
The “most wins” heuristic applied to the first children in tic-tac-toe.



Three wins through a corner square



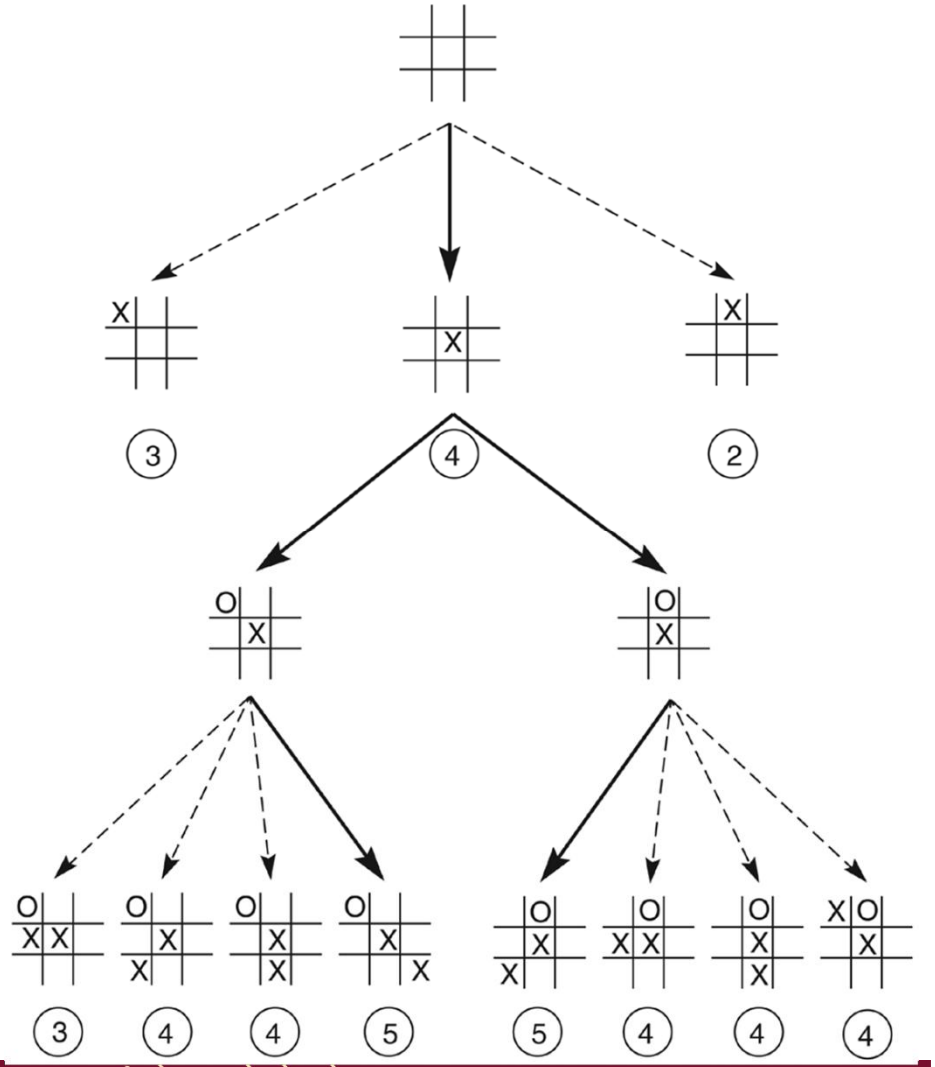
Four wins through the center square



Two wins through a side square

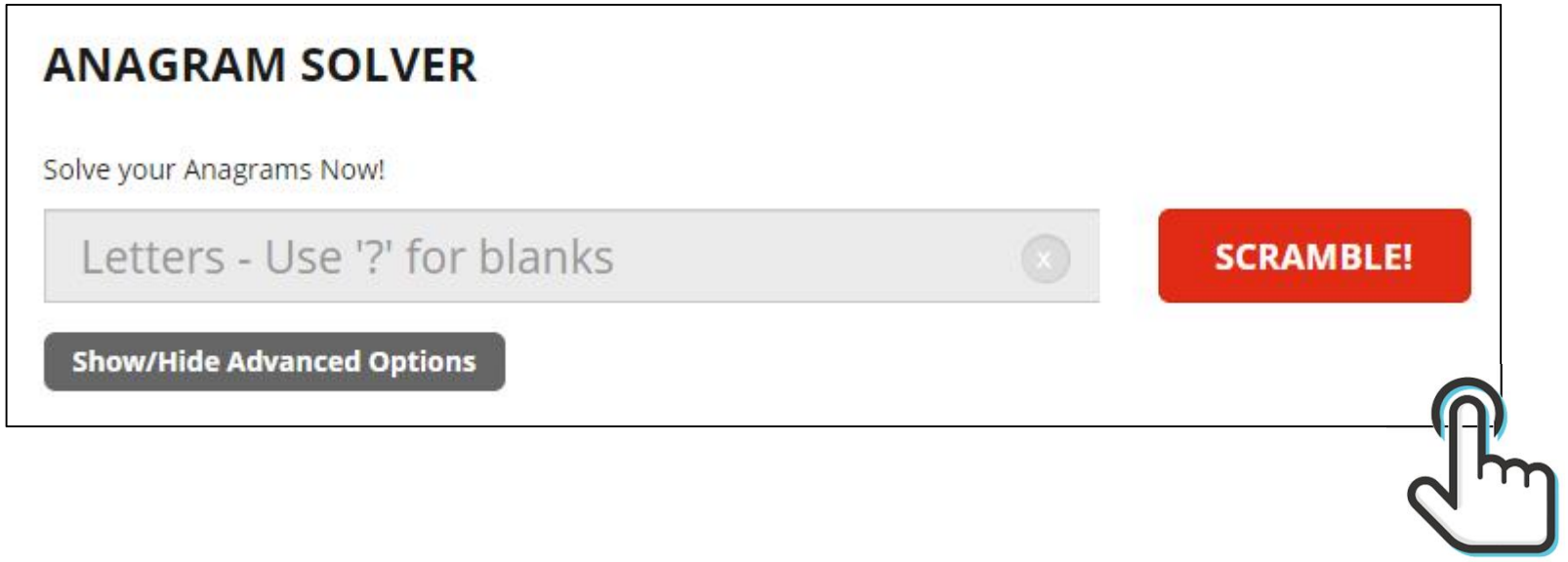
Direct Heuristic Search: Intro

Heuristically reduced state space for tic tac toe.



Direct Heuristic Search : Anagram

- A word that is spelled with the exact same letters as another word. Example: RIDES is an anagram of SIRED and vice versa



Breadth First Search (BFS)

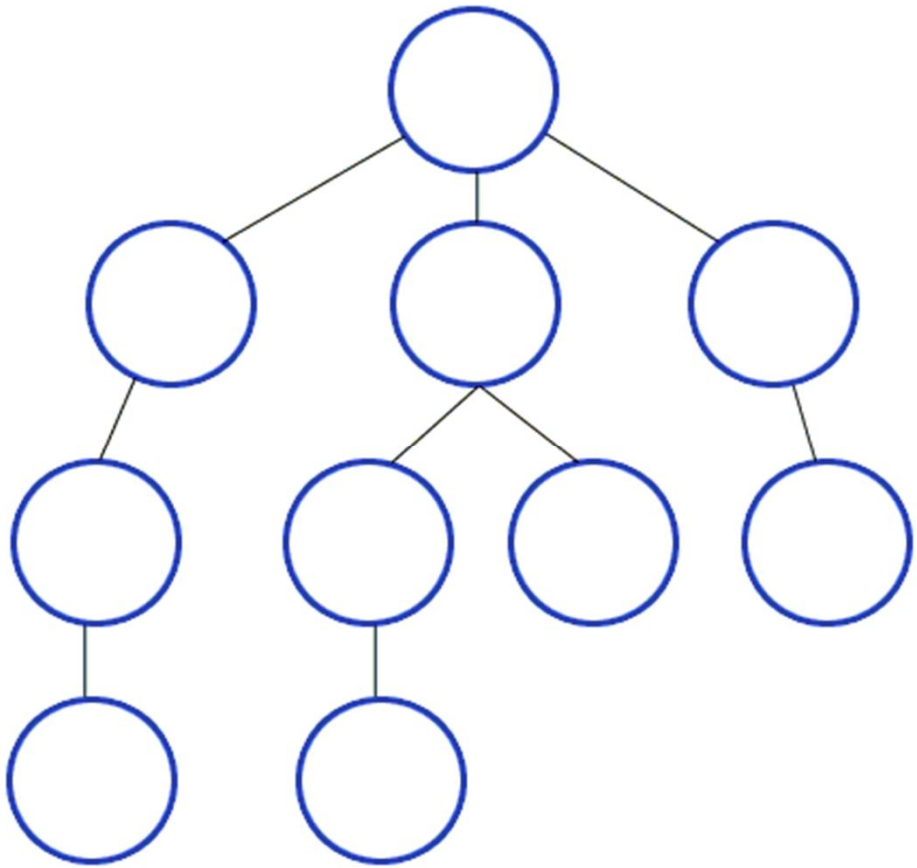
- Breadth-first search (BFS) is a method for exploring a tree or graph. In a BFS, you first explore all the nodes one step away, then all the nodes two steps away, etc.

- Breadth-first search is like throwing a stone in the center of a pond. The nodes you explore "ripple out" from the starting point.

- BFS is a traversing algorithm where you should start traversing from a selected node (source or starting node) and traverse the graph layer wise thus exploring the neighbor nodes (nodes which are directly connected to source node). You must then move towards the next-level neighbor nodes.

- As the name BFS suggests, you are required to traverse the graph breadthwise as follows:
  1. First move horizontally and visit all the nodes of the current layer
  2. Move to the next layer

Breadth First Search (BFS)



Advantages:

- A BFS will find the **shortest path** between the starting point and any other reachable node. A depth-first search will not necessarily find the shortest path

Disadvantages

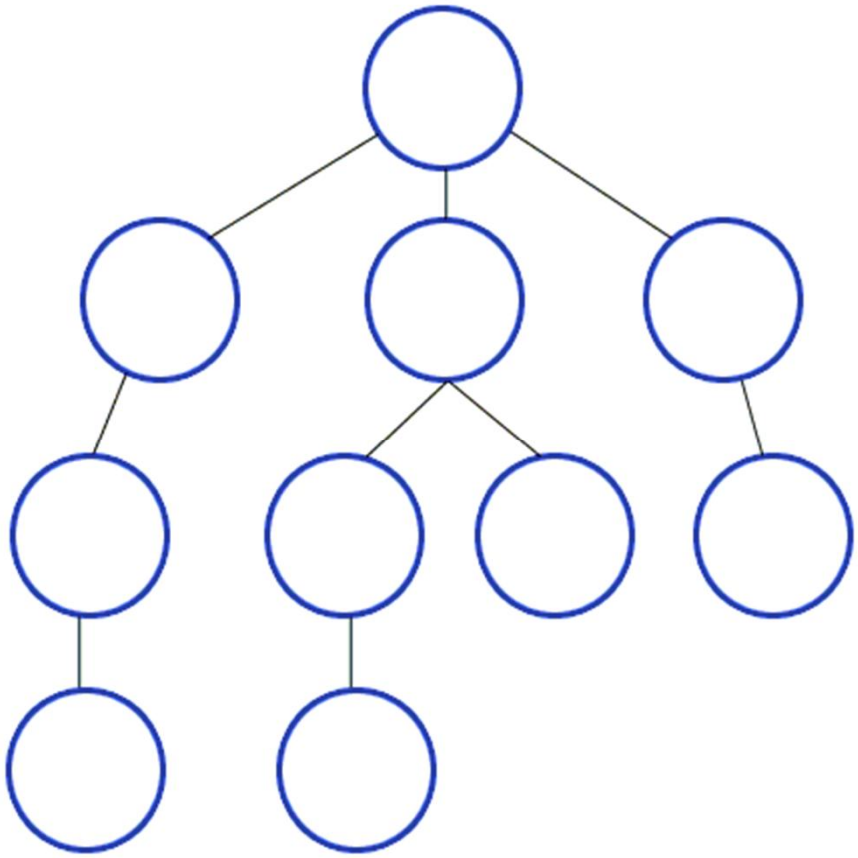
- A BFS on a binary tree *generally* requires more memory than a DFS

Depth First Search (DFS)

- Depth-first search (DFS) is a method for exploring a tree or graph. In a DFS, you go as deep as possible down one path before backing up and trying a different one.

- Depth-first search is like walking through a corn maze. You explore one path, hit a dead end, and go back and try a different one.

Depth First Search (DFS)



Advantages

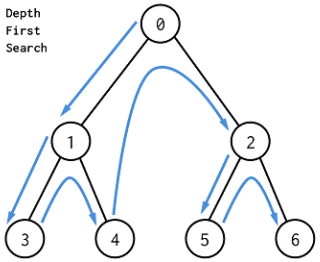
- Depth-first search on a binary tree generally requires less memory than breadth-first
- Depth-first search can be easily implemented with recursion

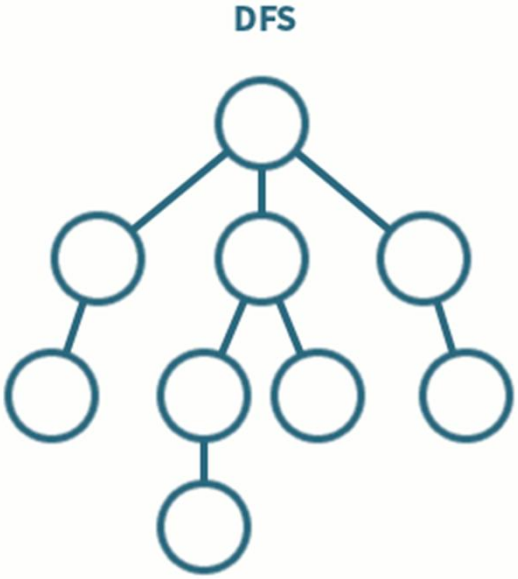
Disadvantages

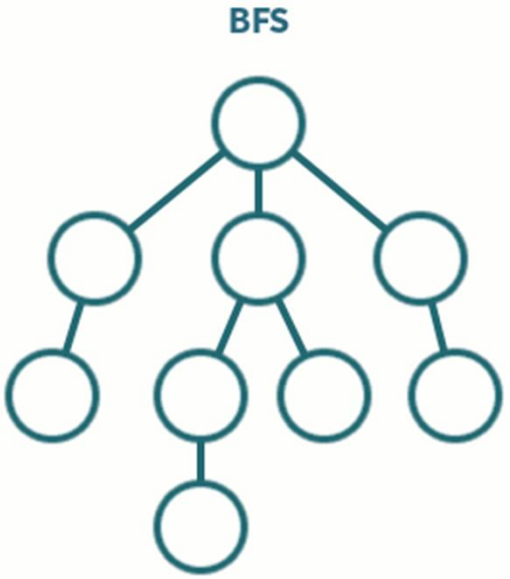
- A DFS doesn't necessarily find the shortest path to a node, while breadth-first search does

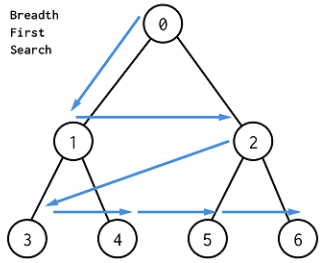
DFS vs BFS

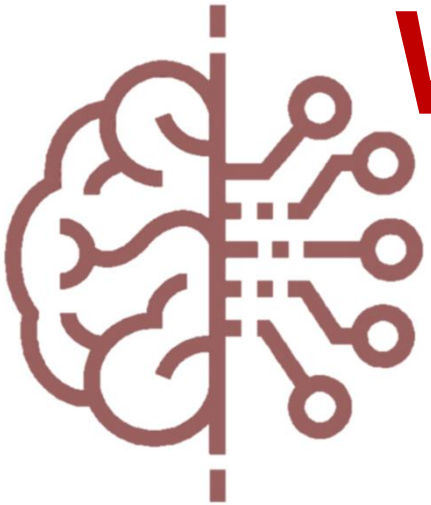
List down all the differences between DFS and BFS











Weak Heuristic Search

- Hill climbing Search
- Best-first Search
- A* Search

Hill Climbing

- Simplest, the best child is selected for further expansion
- Limited memory, no backtracking and recovery

- Problem with hill climbing:
  - An erroneous heuristic can lead along an infinite paths that fail.
  - Can stuck at local maxima reach a state that is better evaluation than its children, the algorithm halts.
  - There is no guarantee optimal performance

- Advantage
  - Can be used effectively if the heuristic is sufficient

Hill Climbing

- Here the generate and test method is augmented by an heuristic function which measures the closeness of the current state to the goal state.
  1. Evaluate the initial state if it is goal state quit otherwise current state is initial state.
  2. Select a new operator for this state and generate a new state.
  3. Evaluate the new state
     1. if it is closer to goal state than current state make it current state
     2. if it is no better ignore
  4. If the current state is goal state or no new operators available, quit. Otherwise repeat from 2.
- In the case of the four cubes a suitable heuristic is the sum of the number of different colours on each of the four sides, and the goal state is 16 four on each side. The set of rules is simply choose a cube and rotate the cube through 90 degrees. The starting arrangement can either be specified or is at random.

Hill Climbing: Type

Simple Hill climbing

- It examines the neighboring nodes one by one and selects the first neighboring node which optimizes the current cost as next node.

Steepest-Ascent Hill climbing

- It first examines all the neighboring nodes and then selects the node closest to the solution state as of next node.

Stochastic hill climbing

- It does not examine all the neighboring nodes before deciding which node to select .It just selects a neighboring node at random and decides (based on the amount of improvement in that neighbor) whether to move to that neighbor or to examine another.

Hill Climbing: Issues

Local Maximum

- All neighboring states have values worse than the current. The greedy approach means we won't be moving to a worse state. This terminates the process even though there may have been a better solution. As a workaround, we use backtracking.

Plateau

- All neighbors to it have the same value. This makes it impossible to choose a direction. To avoid this, we randomly make a big jump.

Ridge

- At a ridge, movement in all possible directions is downward. This makes it look like a peak and terminates the process. To avoid this, we may use two or more rules before testing.

Best First Search

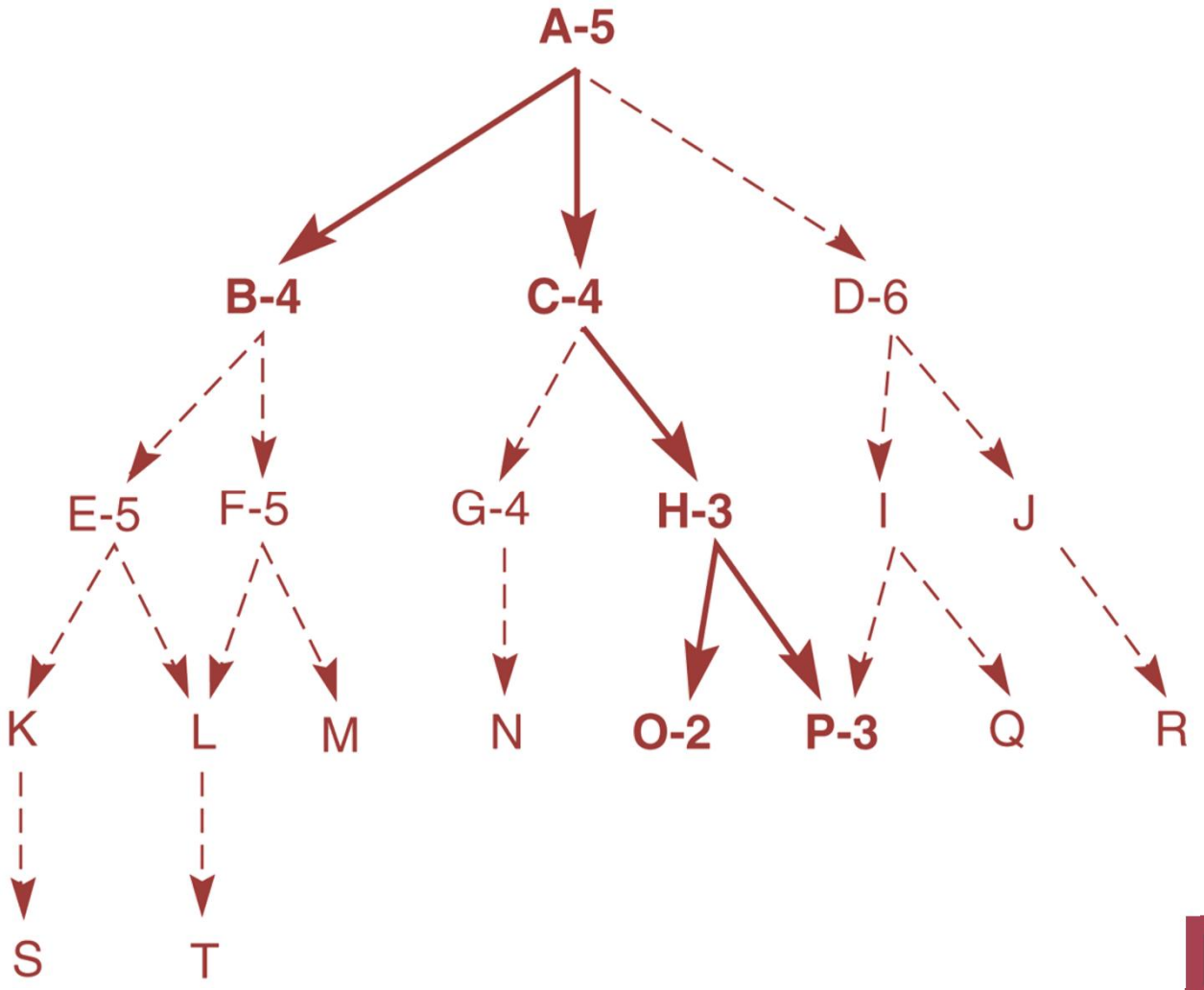
- It is a general algorithm for heuristically searching any state space graph
- Supports a variety of heuristic evaluation functions
- Better and flexible Algorithm for heuristic search
- Avoid local maxima, dead ends; has open and close lists
- selects the most promising state
- apply heuristic and sort the 'best' next state in front of the list (priority queue) can jump to any level of the state space
- If lead to incorrect path, it may retrieve the next best state

Best First Search


```
function best_first_search;

begin
   open := [Start];                                          % initialize
   closed := [ ];
   while open ≠ [ ] do                                       % states remain
      begin
         remove the leftmost state from open, call it X;
         if X = goal then return the path from Start to X
         else begin
                  generate children of X;
                  for each child of X do
                  case
                        the child is not on open or closed:
                           begin
                                 assign the child a heuristic value;
                                 add the child to open
                           end;
                        the child is already on open:
                           if the child was reached by a shorter path
                           then give the state on open the shorter path
                        the child is already on closed:
                           if the child was reached by a shorter path then
                              begin
                                 remove the state from closed;
                                 add the child to open
                              end;
                  end;                                       % case
                  put X on closed;
                  re-order states on open by heuristic merit (best leftmost)
               end;
return FAIL                                                  % open is empty
end.
```

Best First Search



Best First Search

open = [A5]; closed = [ ]

evaluate A5; open = [B4,C4,D6]; closed = [A5]

evaluate B4; open = [C4,E5,F5,D6]; closed = [B4,A5]

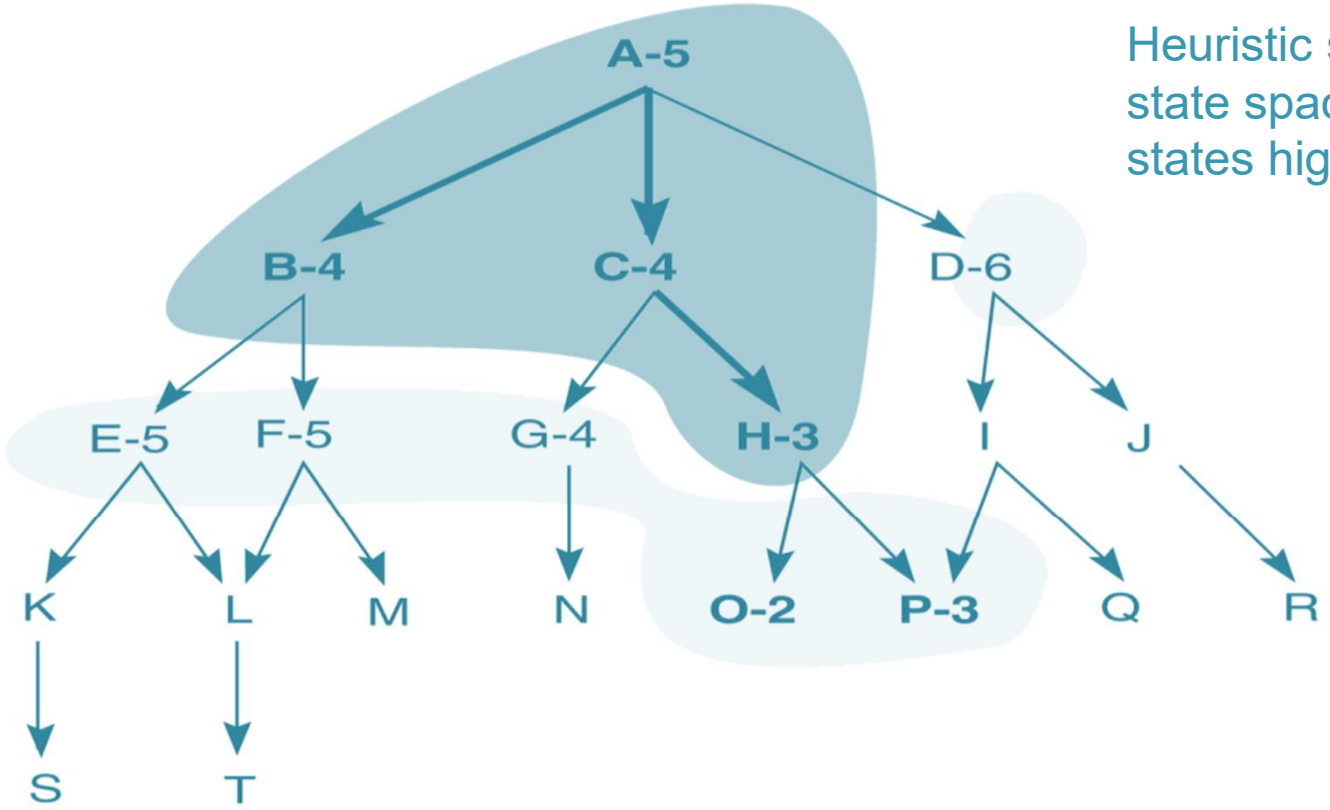
evaluate C4; open = [H3,G4,E5,F5,D6]; closed = [C4,B4,A5]

evaluate H3; open = [O2,P3,G4,E5,F5,D6]; closed = [H3,C4,B4,A5]

evaluate O2; open = [P3,G4,E5,F5,D6]; closed = [O2,H3,C4,B4,A5]

evaluate P3; the solution is found!

Best First Search



Heuristic search of a hypothetical state space with open and closed states highlighted.

A* Search

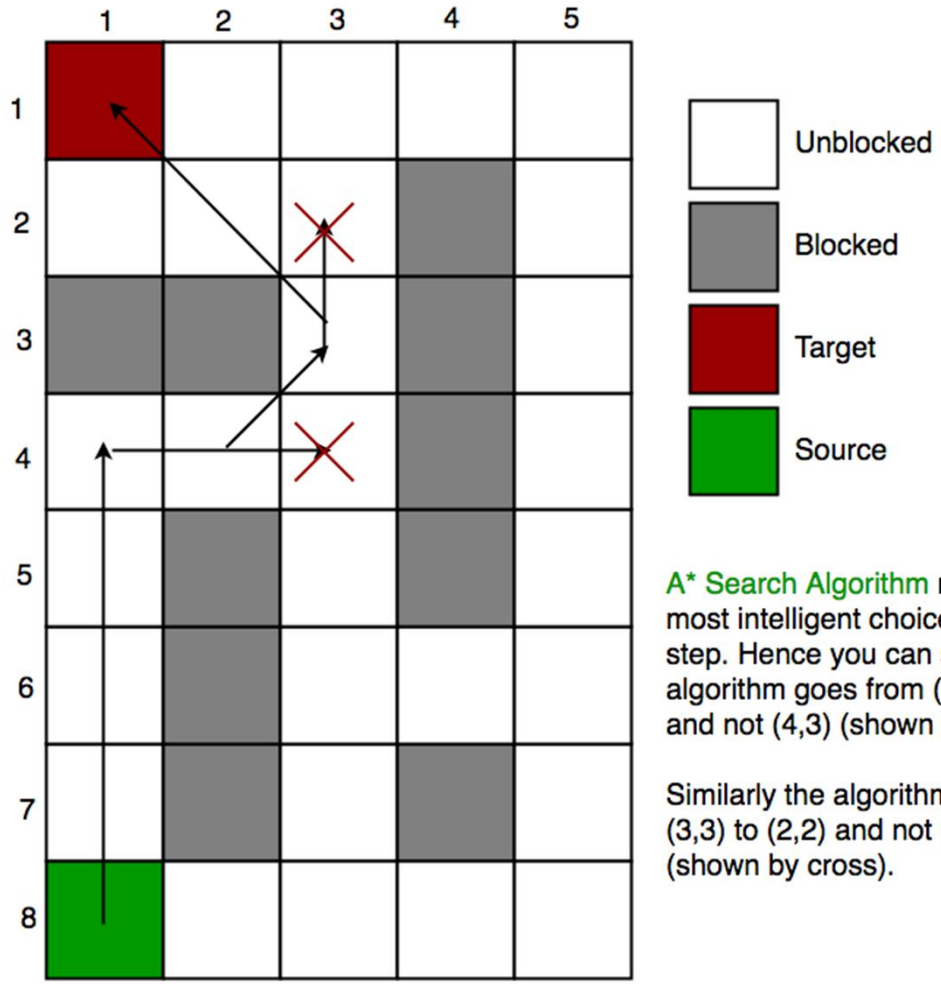
What is A* Search Algorithm?

- A* Search algorithm is one of the best and popular technique used in path-finding and graph traversals.

Why A* Search Algorithm?

- Informally speaking, A* Search algorithms, unlike other traversal techniques, it has “brains”. What it means is that it is really a smart algorithm which separates it from the other conventional algorithms.
- Many games and web-based maps use this algorithm to find the shortest path very efficiently (approximation).

A* Search



A* Search Algorithm makes the most intelligent choice at each step. Hence you can see that algorithm goes from (4,2) to (3,3) and not (4,3) (shown by cross).

Similarly the algorithm goes from (3,3) to (2,2) and not (2,3) (shown by cross).

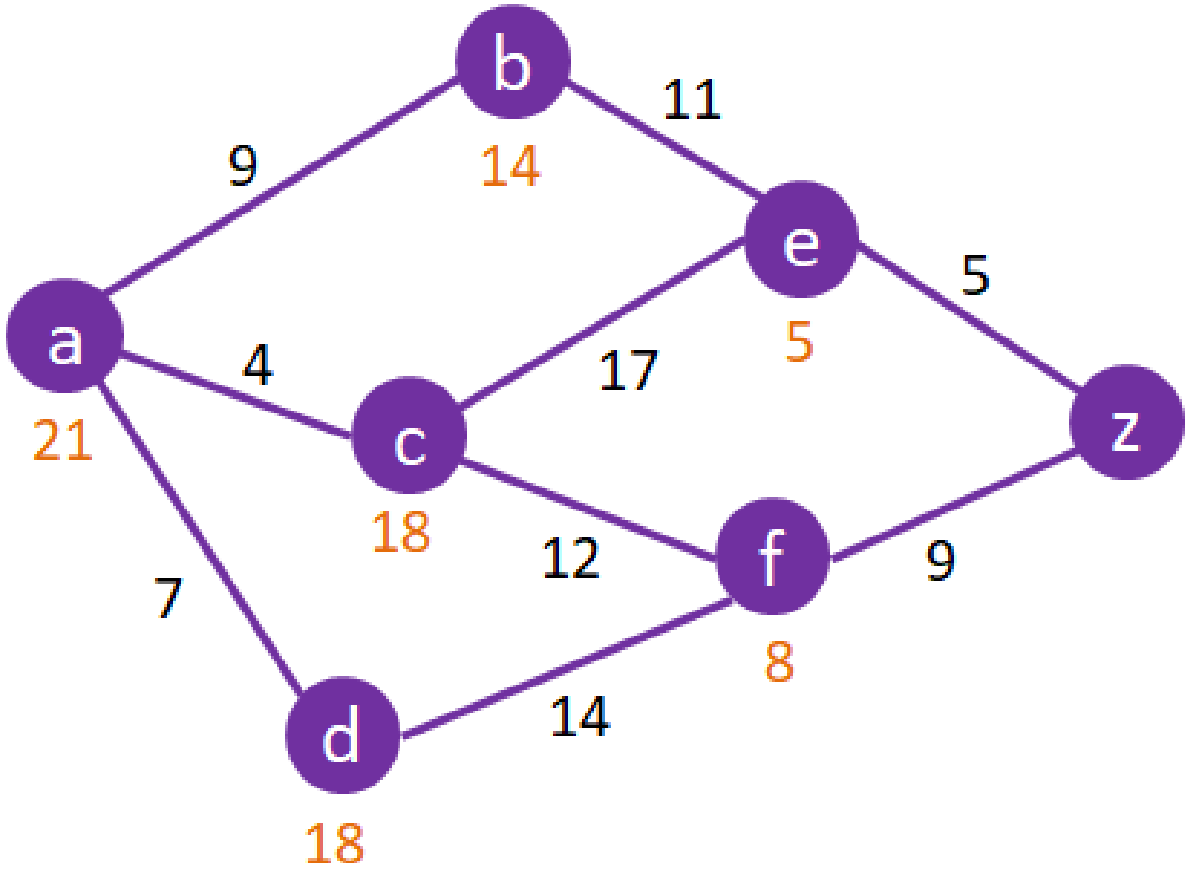
Approximation Heuristics

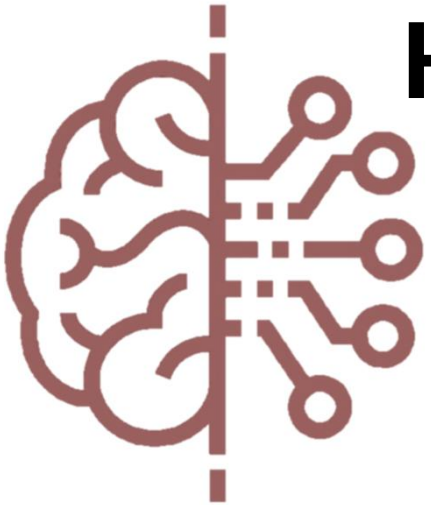
There are generally three approximation heuristics to calculate h

1. Manhattan Distance
2. Diagonal Distance
3. Euclidean Distance

A* Search

What is the shortest path to travel from a to z?





Heuristic Evaluation Function

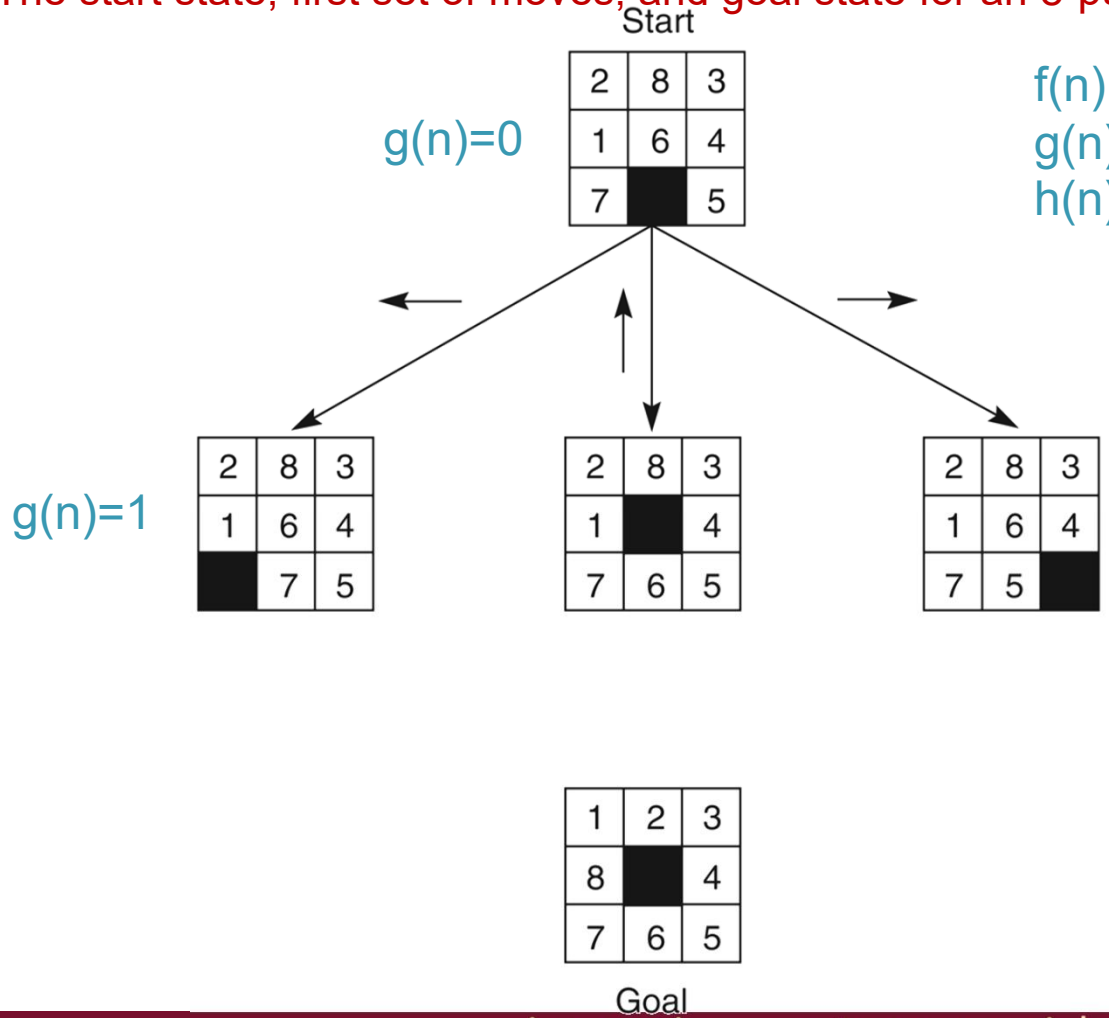
f(n)

Heuristic Evaluation Function f(n)

- To evaluate performances of heuristics for solving a problem
- Devise good heuristic using limited information to make intelligent choices
- To better heuristic, f(n)=g(n)+h(n), where h(n) distance from start to n, g(n) is distance from n to goal
- Eg. 8 puzzle, heuristics h(n) could be:
  - No. of tiles in wrong position
  - No. of tiles in correct position
  - Number of direct reversal (2X)
  - Sum of distances out of place
- And g(n) is the depth measure

Heuristic Evaluation Function f(n)

The start state, first set of moves, and goal state for an 8-puzzle instance.



f(n) = g(n) + h(n)
g(n) = actual dist. From n to start
h(n) = no. of tiles in wrong position

Activity: Heuristic Evaluation Function f(n)

Three heuristics applied to states in the 8 puzzle

- Devising good heuristics is sometimes difficult; OUR GOAL is to use the limited information available to make INTELLIGENT CHOICE

g(n) = ?	2 8 3 1 6 4 ■ 7 5	?	?	0
g(n) = ?	2 8 3 1 ■ 4 7 6 5	?	?	0
g(n) = ?	2 8 3 1 6 4 7 5 ■	?	?	0
		Tiles out of place h(n)	Sum of distances out of place f(n)	2 x the number of direct tile reversals

Use formula as follows:
f(n) = g(n) + h(n)
g(n) = actual dist. From n to start
h(n) = no. of tiles in wrong position

1	2	3
8	■	4
7	6	5

Goal

Premier Digital Tech University ™

Heuristic Evaluation Function f(n)

Three heuristics applied to states in the 8 puzzle

- Devising good heuristics is sometimes difficult; OUR GOAL is to use the limited information available to make INTELLIGENT CHOICE

		Tiles out of place	Sum of distances out of place	2 x the number of direct tile reversals
		h(n)	f(n)	
g(n) = 1	2 8 3 / 1 6 4 / ■ 7 5	5	6	0
g(n) = 2	2 8 3 / 1 ■ 4 / 7 6 5	3	4	0
g(n) = 3	2 8 3 / 1 6 4 / 7 5 ■	5	6	0

Use formula as follows:
f(n) = g(n) + h(n)
g(n) = actual dist. From n to start
h(n) = no. of tiles in wrong position

1	2	3
8	■	4
7	6	5

Goal

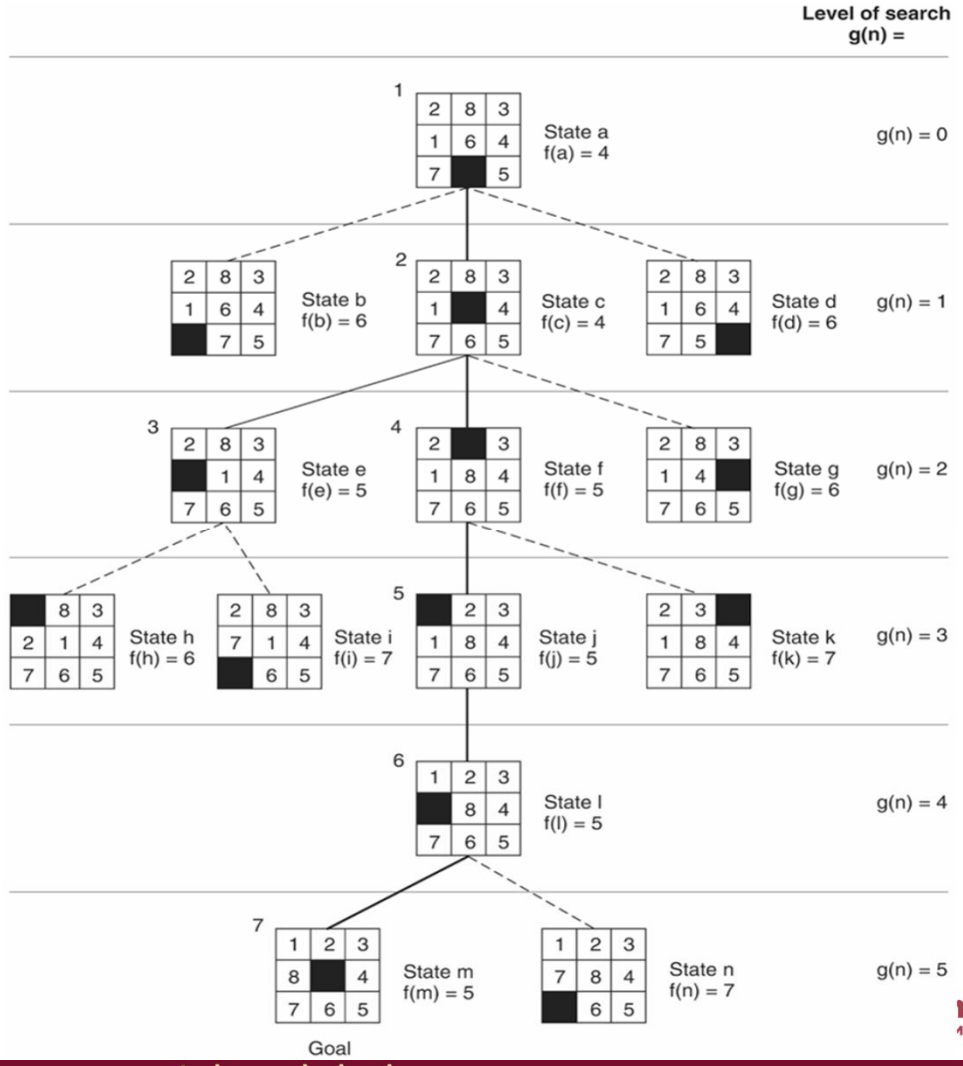
Activity: Open & Closed states

- In the tree of 8 puzzle given in the next slide, Give the value of f(n) for each state, based on g(n) and h(n)

  f(n) = g(n) + h(n)
  g(n) = actual dist. From n to start
  h(n) = no. of tiles in wrong position

- Trace using best first search, what will be the lists of open and closed states? Complete the lists below until successfully achieving the goal.

1. **open = [a4];**
   closed = [ ]
2. **open = [c4, b6, d6];**
   closed = [a4]
3. **open = [e5, f5, b6, d6, g6];**
   closed = [a4, c4]

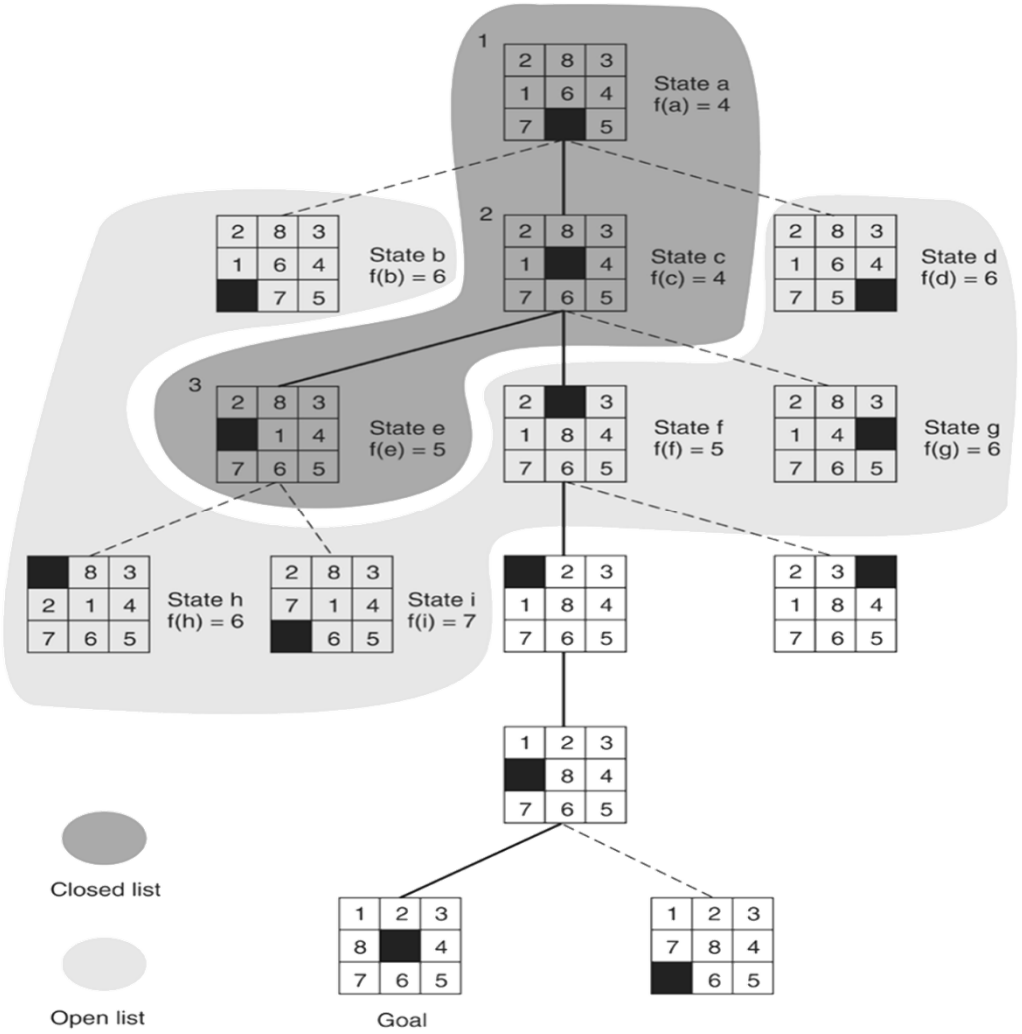


Answer: Open & Closed states

- The successive stages of open and closed that generate previous graph are:

1. **open = [a4];**
   closed = [ ]
2. **open = [c4, b6, d6];**
   closed = [a4]
3. **open = [e5, f5, b6, d6, g6];**
   closed = [a4, c4]
4. **open = [f5, h6, b6, d6, g6, l7];**
   closed = [a4, c4, e5]
5. **open = [j5, h6, b6, d6, g6, k7, l7];**
   closed = [a4, c4, e5, f5]
6. **open = [l5, h6, b6, d6, g6, k7, l7];**
   closed = [a4, c4, e5, f5, j5]
7. **open = [m5, h6, b6, d6, g6, n7, k7, l7];**
   closed = [a4, c4, e5, f5, j5, l5]
8. **success, m = goal!**

Activity: Evaluate f(n)



Open and closed as they appear after the third iteration of heuristic search.

Evaluate why from the current state e5, this algorithm moves to f5 but not i7 in the next step.

Premier Digital Tech University ™